

Forecasting Household Appliance Energy Use: a Case Study Comparing Benchmark, SARIMA, Machine-Learning and Foundation-Model Forecasts

Time Series Case Study — Austin Kauba

Code: [github repository \(src/, scripts/, notebooks/, tests/\)](#) — fully reproducible via `scripts/run_pipeline.py`

1. Introduction

Short-term forecasts of household electricity demand underpin smart-home automation, demand response and home battery scheduling. This case study asks a deliberately practical question: when forecasting the next 24 hours of appliance energy use, how much accuracy do progressively more complex models actually buy? I compare five naive benchmarks, a seasonal ARIMA model selected by an exhaustive AIC grid search, a gradient-boosted tree model (XGBoost) with engineered covariates, and a zero-shot time-series foundation model (Chronos-2), all under an identical rolling-origin evaluation. Following the guidance of Hyndman & Athanasopoulos (2021), every model must justify itself against the strongest simple benchmark, not merely against other complex models.

2. Data and preprocessing

The Appliances Energy Prediction dataset (Candanedo et al., 2017; UCI ML Repository) records a low-energy house in Stambruges, Belgium, from 11 January to 27 May 2016 at 10-minute resolution: 19,735 rows containing appliance energy use (Wh), light energy, nine indoor temperature/humidity sensor pairs (T1-T9, RH_1-RH_9) and outdoor weather from Chievres airport. Data-quality checks found no missing values, no duplicate timestamps and no gaps in the 10-minute grid, so no imputation was needed.

The series was resampled to hourly resolution (3,290 complete hours) by averaging the six samples in each hour. Averaging rather than summing keeps the target on the raw scale, and since every hour has exactly six samples the two differ only by a factor of six — model rankings are unaffected. Hourly resolution was chosen because it makes seasonal ARIMA with a daily period ($m = 24$) computationally tractable, whereas $m = 144$ at 10-minute resolution would not be, and because a 24-hour-ahead appliance forecast does not plausibly require sub-hourly granularity. The hourly target averages 97.8 Wh (median 63.3) with strong right skew (max 608 Wh): the house idles at a low baseline punctuated by cooking/laundry spikes, a shape that will matter repeatedly in what follows.

3. Exploratory analysis and stationarity

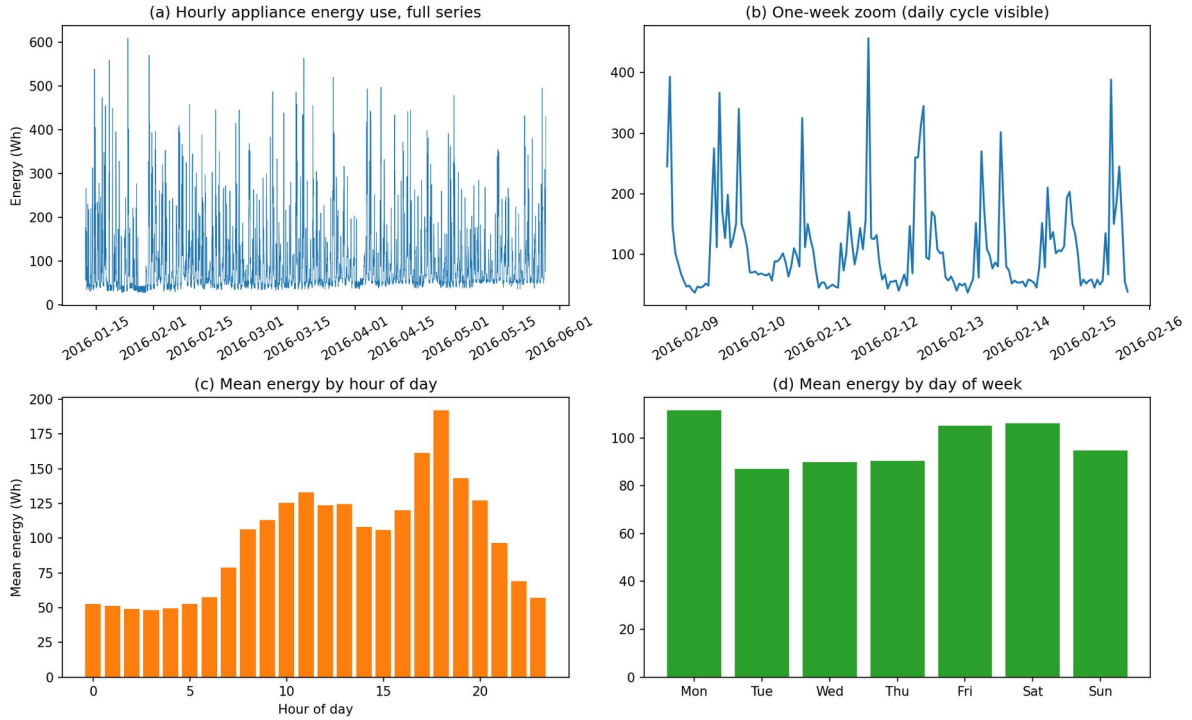


Figure 1. Hourly appliance energy use: full series (a), one-week zoom (b), and mean profiles by hour of day (c) and day of week (d).

Figure 1 shows the dominant structure: a daily cycle with a quiet overnight baseline (~ 45 Wh), a midday shoulder and an evening peak around 18:00 (~ 150 Wh on average), plus a mild weekly effect (weekend use is higher). There is no visible long-term trend over the 4.5 months. STL decomposition quantifies this: seasonal strength (Hyndman & Athanasopoulos, 2021) is 0.32 at the daily period and 0.37 at the weekly period, while trend strength is only 0.05. Seasonality is therefore real but moderate — most of the variance sits in an irregular, spiky remainder driven by discrete human activities, which no deterministic seasonal pattern can predict.

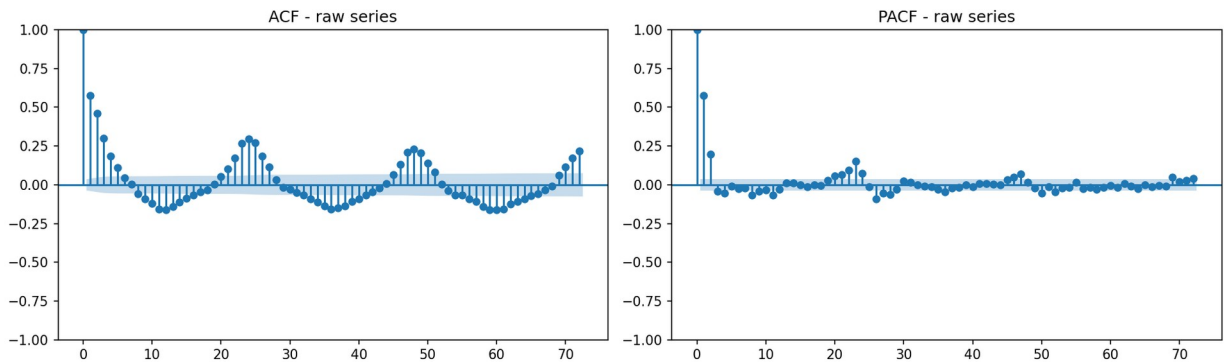


Figure 2. ACF and PACF of the raw hourly series out to 72 lags. Spikes at lags 24, 48 and 72 reveal the daily seasonal component.

Stationarity was assessed on the training period with complementary tests. The ADF test strongly rejects a unit root (statistic -8.76 , $p < 0.001$) and KPSS fails to reject level-stationarity (statistic 0.061 , $p \geq 0.1$): the series is already stationary in level, so no Box-Cox transformation or differencing is strictly required. Both tests agree after first and seasonal differencing as well (Table 1). The ACF (Figure 2), however, decays slowly with pronounced peaks every 24 lags — stationarity does not mean absence of structure, and the seasonal autocorrelation motivates the seasonal models that follow.

Series	ADF stat	ADF p	KPSS stat	KPSS p	Conclusion
raw	-8.76	0.000	0.061	≥ 0.1	stationary
diff(1)	-15.95	0.000	0.042	≥ 0.1	stationary
seasonal diff(24)	-12.67	0.000	0.015	≥ 0.1	stationary

Table 1. Stationarity tests (train period). ADF H_0 : unit root; KPSS H_0 : stationary.

4. Forecasting design

Target: hourly appliance energy use (Wh). Horizon: 24 hours. Test period: the final 14 days (336 h), held out from all model and hyperparameter selection. Evaluation is rolling-origin: each model issues one 24-hour forecast per day, seeing only data strictly before each forecast origin, exactly as an operational system would run. This yields 336 genuine ≤ 24 h-ahead predictions per model rather than a single noisy 24-point sample. Metrics: MAE, RMSE, MAPE, Bias, and MASE (Hyndman & Koehler, 2006) scaled by the in-sample daily seasonal naive MAE, so $MASE < 1$ means the model beats copying yesterday. MAPE is safe here since hourly use never reaches zero, but it inflates errors at low baseline hours and is reported for completeness rather than ranking.

5. Benchmark models

Five benchmarks were evaluated: the training mean, the naive (last value), daily and weekly seasonal naives (same hour yesterday / last week), and drift. The weekly seasonal naive is the strongest on MAE (43.5 Wh, MASE 0.81) as it captures both the daily shape and weekend behaviour; the daily variant is worse (48.3 Wh) because weekday/weekend days are copied across each other. Strikingly, the flat mean wins RMSE among benchmarks (74.9 vs 81.4 Wh): copying last week also copies its unrepeatable spikes, and RMSE punishes those double-counted spikes heavily, whereas the mean never gambles. Naive and drift fail badly (MASE 1.6) — appliance use mean-reverts within hours, so the most recent value carries almost no information about tomorrow. These results already sketch the data-generating process: a stable daily/weekly rhythm plus unpredictable activity spikes.

6. SARIMA and SARIMAX

Following the AIC-based procedure from the course, I looped over the full grid required by the brief — $p = 0..6$, $d = 0..2$, $q = 0..6$, 147 models — using an `optimize_ARIMA`-style function with checkpointing. The best AICs were essentially tied between $ARIMA(5,1,5)$ (33099.4) and $ARIMA(1,1,3)$ (33099.9); applying a parsimony rule (fewest parameters within 2 AIC of the minimum; Burnham & Anderson, 2004) selects (1,1,3). Note the mild tension with Section 3: the tests say $d = 0$ suffices, while AIC marginally prefers $d = 1$; differencing a stationary series is harmless for forecasting though it can slightly inflate forecast variance. A second stage gridded the seasonal terms $P, D, Q \in \{0,1\}$ at $m = 24$ (a full 0..6 seasonal grid is computationally infeasible and seasonal orders above 1 are rarely warranted; Hyndman & Athanasopoulos, 2021). Seasonal differencing plus SAR/SMA terms improved AIC dramatically, from 33099.9 to 32554.5 — overwhelming evidence for the daily seasonal component. Final model: $SARIMA(1,1,3)(1,1,1,24)$.

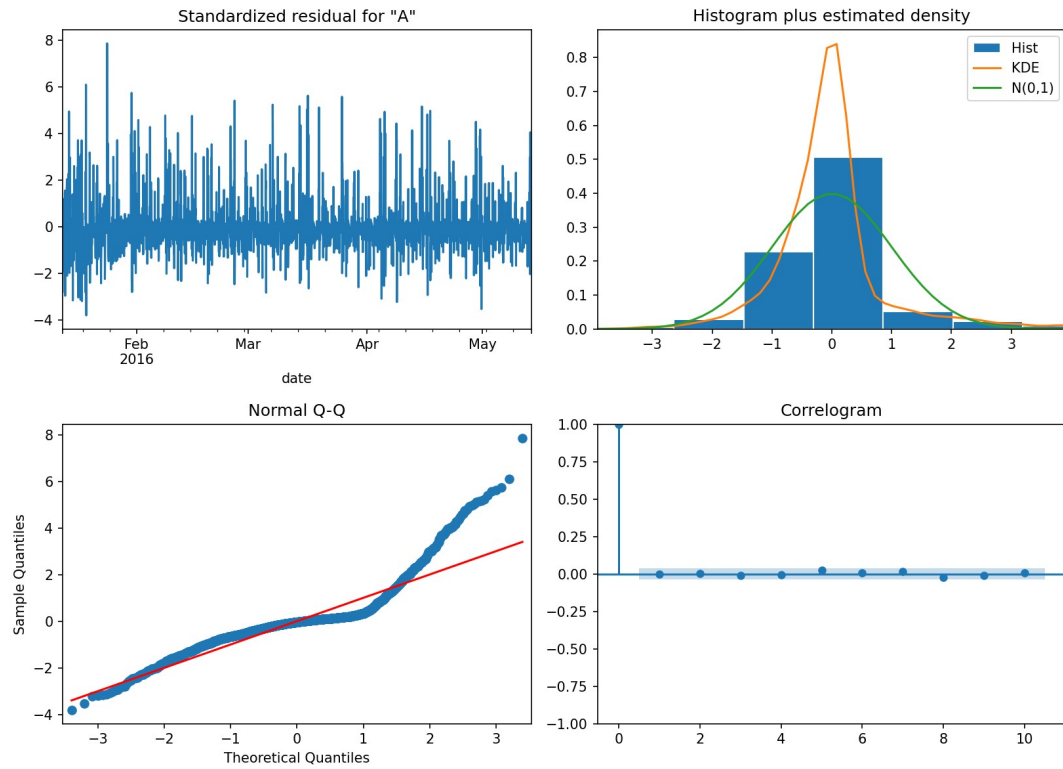


Figure 3. Residual diagnostics for $SARIMA(1,1,3)(1,1,1,24)$: standardised residuals, histogram vs $N(0,1)$, Q-Q plot and correlogram.

Diagnostics (Figure 3) show a flat residual ACF, confirmed by Ljung-Box tests at lags 24 and 48 ($p = 0.18$ and 0.08): the residuals are consistent with white noise, so the autocorrelation and daily seasonality are adequately captured. The histogram and Q-Q plot, however, reveal heavy right tails — the model treats consumption spikes as Gaussian noise, so its 95% intervals are only approximate and its point forecasts regress towards the typical daily profile. Rolling-origin forecasts (parameters fixed, Kalman filter updated daily) achieve MAE 37.9 Wh, RMSE 64.7, MASE 0.71 — a 13% MAE improvement on the strongest benchmark. A SARIMAX variant adding outdoor weather (T_{out} , RH_{out} , Windspeed) and calendar terms was also fitted; its AIC (32562.9) and test accuracy (MAE 38.2) are both marginally worse: once the daily cycle and autocorrelation are modelled, realised weather adds essentially nothing for this house — and since future weather would in reality be an imperfect forecast, the small conditional gain is an upper bound (Section 10, Q5).

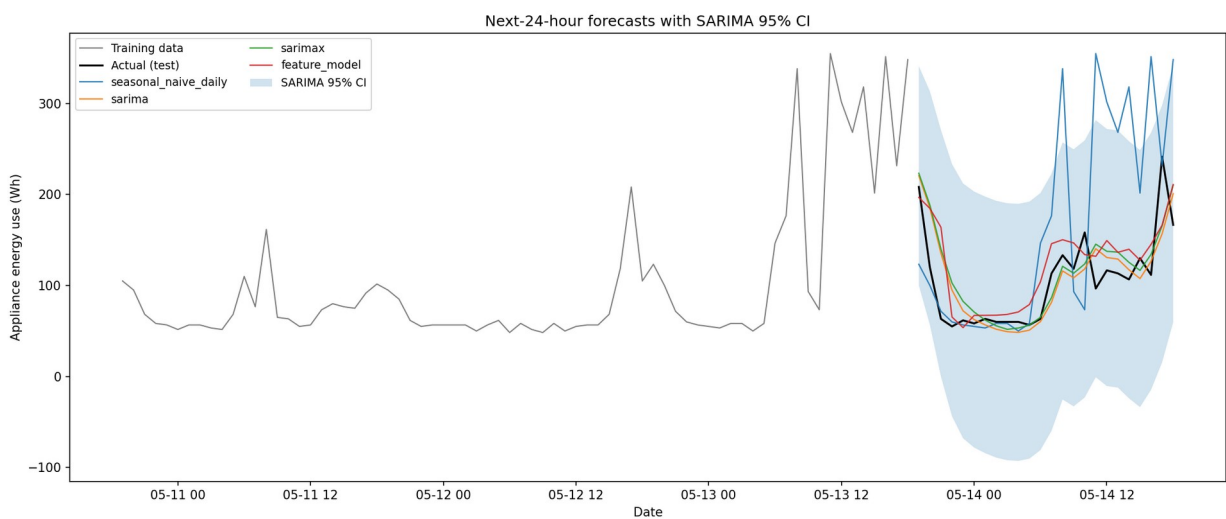


Figure 4. Next-24-hour forecasts for the first test day with the SARIMA 95% confidence interval. The actual series stays within the band, but the band is wide (and crosses zero) — a consequence of Gaussian errors on a skewed series.

7. Feature-based model: XGBoost

A supervised table was built with five feature groups: cyclically-encoded time features (hour, day-of-week, weekend flag); lagged target values (1, 2, 3, 6, 12, 24, 48, 168 h); rolling mean/std/max of the past-only target over 3, 24 and 168 h windows; the nine indoor sensor pairs; and outdoor weather. All lag/rolling features are shift()-ed so features at hour t use only data before t (verified by unit tests). XGBoost (Chen & Guestrin, 2016) was chosen for its strength on tabular data and native nonlinearity/interaction handling; hyperparameters (depth, learning rate, trees) were tuned on the final 14 days of the training period, never the test set. Critically, forecasting is genuinely recursive: within each 24 h window the model's own predictions feed its lag features. Naively predicting test rows directly would let lag₁ use actual test values — a subtle leak that turns a 24 h forecast into a 1 h one.

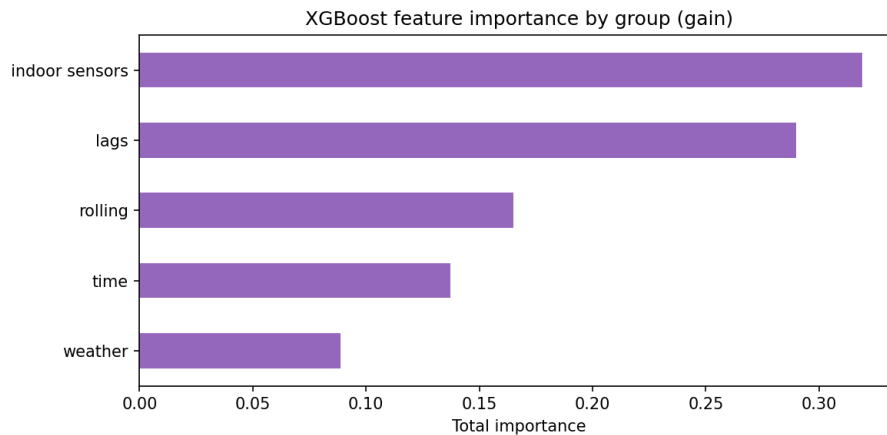


Figure 5. XGBoost feature importance (gain) summed by feature group.

Two variants isolate the covariate question. The full model (all groups, conditional on realised sensor/weather values) achieves MAE 41.9, RMSE 67.0. The past-only variant — lags, rolling and time features alone — is better: MAE 38.6, RMSE 64.4 (the best RMSE of any model), MASE 0.72, and near-zero bias (−0.3 Wh). The grouped importances (Figure 5) need careful reading: the indoor-sensor group has the largest summed gain (0.32), but only because it contains 19 features; per feature, lags dominate (mean gain 0.035 vs 0.017), and lag₁ alone is the single most important feature (0.14), followed by calendar terms and short rolling means. The sensors' diffuse contribution helps in-sample — indoor temperature partly proxies appliance activity (reverse causation) — but does not survive a 24 h recursive forecast, where it evidently adds more variance than signal. Consistent with this, the richer feature set selected deeper trees in tuning (depth 7 vs 3), a symptom of mild overfitting.

8. Foundation model: Chronos-2

Chronos-2 (Ansari et al., 2024) is a pretrained time-series foundation model that tokenises a scaled context window and generates a predictive distribution autoregressively. It was applied zero-shot and target-only: no training on this dataset, no covariates, a 512 h context, and the same rolling-origin protocol (one 24 h quantile forecast per test day, context extended with actuals between days). Its forecast is therefore a true operational forecast, directly comparable to SARIMA and the past-only XGBoost, and it additionally provides calibrated 10–90% quantile bands without any distributional assumption.

[Chronos-2 results pending: run notebooks/06_foundation_model.ipynb (Colab), place fc_foundation_model.csv in outputs/forecasts/, re-run the evaluate stage, and insert MAE/RMSE/MASE, interval coverage and Figure 6 here.]

9. Results and error analysis

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	MASE	Bias (Wh)
SARIMA(1,1,3)(1,1,1,24)	37.94	64.72	35.30	0.71	−7.55
XGBoost (past-only)	38.61	64.35	36.73	0.72	−0.33
SARIMAX (+weather)	38.22	65.49	35.34	0.72	−6.91
XGBoost (all covariates)	41.89	66.95	42.14	0.78	7.36
Chronos-2 (zero-shot)	[pending]	[pending]	[pending]	[pending]	[pending]
Weekly seasonal naive	43.46	81.41	37.35	0.81	−13.16

Daily seasonal naive	48.31	85.56	43.33	0.90	1.75
Mean	50.26	74.94	53.70	0.94	-3.29
Naive	85.55	110.39	112.91	1.60	50.98
Drift	85.80	110.68	113.31	1.61	51.37

Table 2. Rolling-origin 24 h forecast accuracy over the 14-day test period (best value per column among fitted models in bold context; MASE scale: in-sample daily seasonal naive).

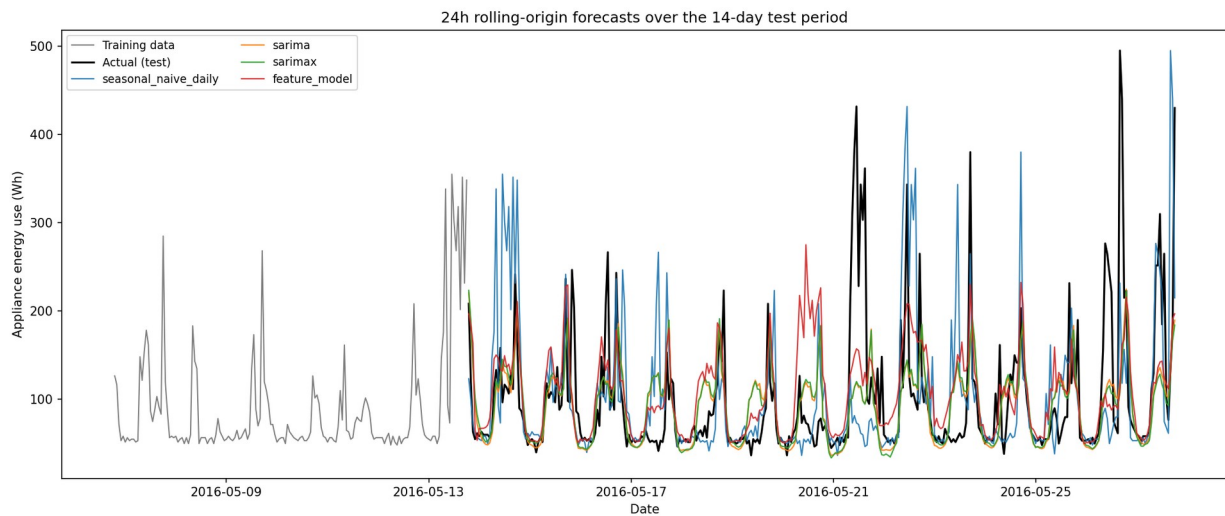


Figure 7. Rolling-origin 24 h forecasts over the 14-day test period (strongest benchmark and main models).

Figure 7 makes the shared failure mode visible: every model tracks the daily rhythm well but truncates the large evening spikes (e.g. 21 and 26 May reach 400–500 Wh while forecasts stay below ~220 Wh). Those spikes are discrete occupant decisions — cooking, laundry — that are simply not predictable from the series' own past, which is why even the best MASE is ~0.7 rather than ~0.3. The seasonal naive, by contrast, sometimes places a copied spike on a day where none occurs, the worst of both worlds under RMSE.

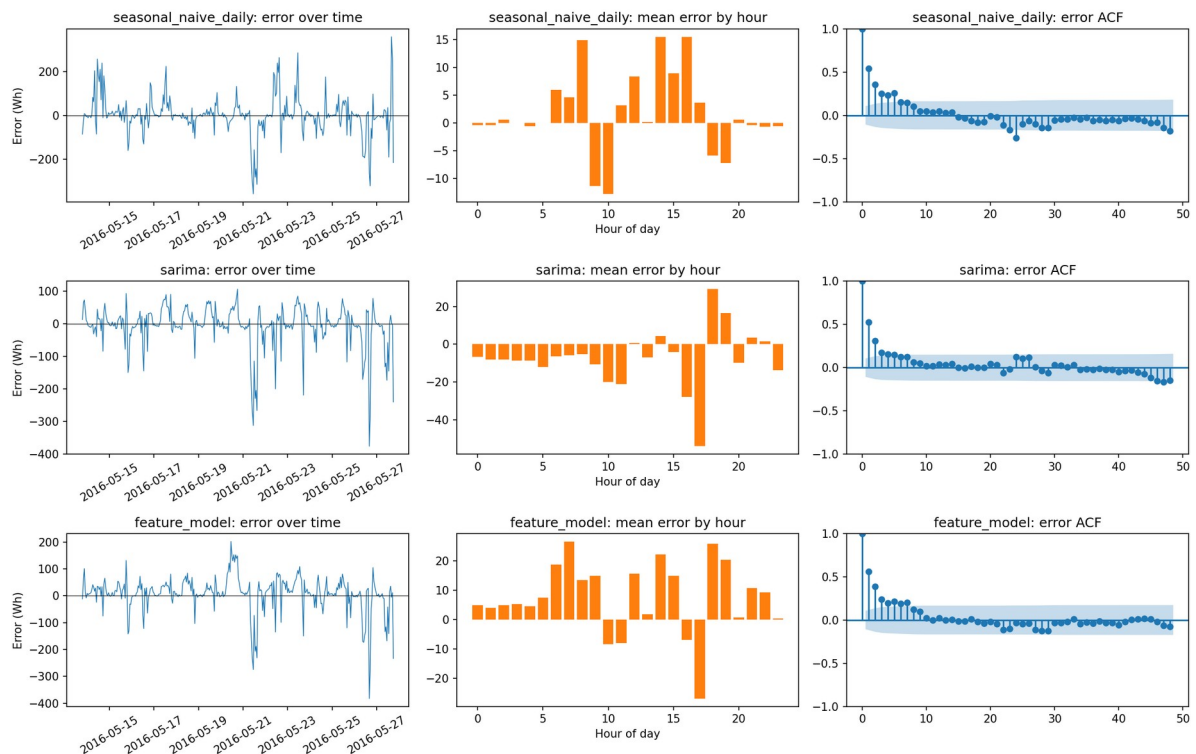


Figure 8. Error diagnostics: error over time, mean error by hour of day, and error ACF for the strongest benchmark and the main models.

The diagnostics in Figure 8 add nuance. SARIMA and XGBoost errors are centred, with hour-of-day bias concentrated in the evening (under-forecast at 17:00–20:00, when spikes cluster) — SARIMA's overall bias of -7.6 Wh reflects this spike truncation, while past-only XGBoost is nearly unbiased. Residual ACFs of the fitted

models are close to white noise at short lags, with small daily-lag remnants indicating some day-to-day amplitude variation that fixed seasonal structure cannot absorb. The seasonal naive's error ACF, by contrast, shows strong structure — exactly the signal the fitted models successfully exploit.

10. Answers to the assignment questions

Q1. Which benchmark is strongest, and what does that say about the data?

The weekly seasonal naive is strongest on MAE/MASE (43.5 Wh, 0.81), beating the daily seasonal naive — appliance use has both daily and weekly (weekend) structure. On RMSE the mean wins among benchmarks, because seasonal copying duplicates one-off spikes. Naive and drift fail completely. Together this says: appliance energy use is a mean-reverting, doubly-seasonal process whose remaining variance is spiky and largely unpredictable — structure worth modelling, but with a hard noise floor.

Q2. Does SARIMAX improve on the strongest seasonal benchmark?

Yes. SARIMA(1,1,3)(1,1,1,24) improves MAE by 13% and RMSE by 21% over the weekly seasonal naive, and Ljung-Box confirms the residuals are white noise, so daily seasonality and short-lag autocorrelation are adequately captured. Weekly seasonality is not explicitly modelled ($m = 168$ is impractical in SARIMA) and survives faintly in the error diagnostics. Exogenous weather variables do not help: AIC worsens and test accuracy is flat — the covariates are adequately proxied by the seasonal structure itself.

Q3. Do the engineered features help the ML model, and which groups matter?

Lag, rolling and time features carry virtually all the usable signal: the past-only XGBoost matches SARIMA (MASE 0.72 vs 0.71) with the best RMSE and near-zero bias. Adding sensor and weather covariates makes the model worse (MAE 41.9 vs 38.6): although the 19 sensor features jointly absorb the largest share of in-sample gain, their per-feature contribution is half that of the lags, indoor temperature is partly a consequence of appliance use rather than a predictor of it, and the extra dimensions promote overfitting that shows up out of sample. The most useful individual features are lag_1, the calendar terms, and short rolling means.

Q4. Does the foundation model outperform, and is it worth the complexity?

[To finalise after the Colab run. Framing: Chronos-2 is zero-shot — no training, no feature engineering — so even matching SARIMA/XGBoost (MASE ≈ 0.7) at $\sim 120M$ parameters and higher inference cost would be remarkable scientifically but hard to justify operationally unless its calibrated quantile bands are needed.]

Q5. Which variables are genuinely known at the forecast origin?

At origin t , known inputs are: the full history of the target and sensors up to t , and all future calendar variables (hour, day-of-week, weekend) — these are deterministic. Future indoor temperature/humidity are unknown (they partly depend on the appliances being forecast); future outdoor weather is unknown too, though a weather forecast could stand in with added error. SARIMAX and the full XGBoost use realised test-period covariate values, so their outputs are conditional forecasts — scenario answers ('given this weather...') — not true operational forecasts, and their measured accuracy is an upper bound. The benchmarks, SARIMA, past-only XGBoost and Chronos-2 are true forecasts. Notably, the conditional models are not better here, so the distinction costs us nothing in this application.

Q6. Which model for practical smart-home forecasting?

The past-only XGBoost, with the daily seasonal naive retained as a fallback sanity check. It ties for best accuracy while being a true operational forecast; it is unbiased; it retrains in seconds and predicts in milliseconds on embedded-class hardware; and tree ensembles offer usable interpretability via feature importances. SARIMA is equally accurate and provides analytic uncertainty intervals, but its Gaussian bands are poorly calibrated for a skewed series (Figure 4) and refitting is minutes rather than seconds. Chronos-2's quantile output is attractive for uncertainty-aware automation, but a $\sim 120M$ -parameter model is heavy for a smart-home device unless served from the cloud.

11. Limitations and future improvements

This study covers one house over 4.5 months (January–May), so weekly patterns rest on ~ 19 weekends, no summer or holiday behaviour is observed, and conclusions may not transfer across households. The headline metrics are point-forecast metrics; a fuller evaluation would score the predictive distribution (CRPS, pinball

loss) — precisely where Chronos-2 and quantile gradient boosting should shine, since the Gaussian SARIMA intervals are visibly miscalibrated for this skewed target (Hong & Fan, 2016). The dominant error source — occupant activity spikes — is invisible to every model used here; occupancy sensing, per-appliance submetering (NILM), or explicit spike/regime models (e.g. Markov-switching, or a two-part baseline+spike model) are the most promising route to breaking the $MASE \approx 0.7$ floor. Further natural extensions: TBATS or Fourier-term dynamic harmonic regression to model dual seasonality explicitly ($m = 24$ and 168); using forecast weather rather than realised weather to quantify the conditional-forecast gap honestly; ensembling SARIMA with XGBoost (their errors are imperfectly correlated); and fine-tuning Chronos-2 on the target series rather than using it zero-shot.

12. Conclusion

Appliance energy use in this household is a stationary, doubly-seasonal, spike-contaminated series. Modelling its structure pays: SARIMA and a past-only XGBoost both beat the strongest benchmark by $\sim 13\%$ MAE and reach $MASE \approx 0.71$. But the more instructive findings are negative: realised weather and indoor-sensor covariates add nothing once seasonality and autocorrelation are modelled, and no model can anticipate discrete occupant activity from the series' past alone. For deployment I recommend the past-only XGBoost for accuracy, speed and honesty (it uses only information genuinely available at the forecast origin), with future work directed at probabilistic evaluation and occupancy-aware spike modelling rather than at further generic model complexity.

References

- Ansari, A. F., Stella, L., Turkmen, C., et al. (2024). Chronos: Learning the language of time series. *Transactions on Machine Learning Research*.
- Burnham, K. P., & Anderson, D. R. (2004). Multimodel inference: understanding AIC and BIC in model selection. *Sociological Methods & Research*, 33(2), 261–304.
- Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81–97.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD '16*, 785–794.
- Hong, T., & Fan, S. (2016). Probabilistic electric load forecasting: a tutorial review. *International Journal of Forecasting*, 32(3), 914–938.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.
- Seabold, S., & Perktold, J. (2010). statsmodels: econometric and statistical modeling with Python. *Proceedings of SciPy 2010*.